# Discovery Executive Summary

**Project:** discovery-27july2026 · **Generated:** 27/07/2026, 13:04:43

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 4 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

## Portfolio Overview

| # | Analysis | Overall Rating | Hotspot Score |
|---|---|---|---|
| 1 | Testing & Quality Assurance Analysis | HIGH RISK | — |
| 2 | Security Analysis | HIGH RISK | — |
| 3 | Performance & Sustainability Analysis | HIGH RISK | — |
| 4 | Technical Debt | MODERATE | — |

---

## 1. Testing & Quality Assurance Analysis

OVERALL CODEBASE RATING — TESTING & QUALITY ASSURANCE

### High Risk

Driven by zero-tested critical logic (H1), ~2% coverage (H2), and the total absence of integration, contract, frontend and E2E tests (H3, H4, H7).

EXECUTIVE SUMMARY

The repository ships with a test suite that exists in name only: **2 PHPUnit files** guard a codebase of **36 source files** across three layers (Laravel backend, Node/Express `dev-api`, React frontend). Both existing tests are vacuous — one asserts a hard-coded literal array equals itself, the other loops over `random_int()` and asserts a probabilistic threshold — so the **effective coverage of real business logic is ~2% (estimated)**; there is no coverage tooling (`coverage: none` in CI) to measure it. Every business-critical module is untested: the IVR-traversal / reachability engine (`RealTimeTestService`, `dev-api/src/realtime.js`), the Mongo persistence layer (`MongoService`), the unsafe-`extract()` `LegacyDataMapper`, and all six API controllers exposing ~18 endpoints. There are **no integration tests** exercising the DB/Mongo/HTTP/SSE boundaries, **no contract tests** for any public API, and **no frontend or end-to-end tests at all** despite a React app with routing, forms, and live SSE feeds. CI runs

`phpunit` and a frontend `build` on every PR, but with the suite this thin the gate provides false confidence rather than protection. Overall testing health is **High Risk** and must be raised before any refactor or extraction work begins.

## 5.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | GOOD | MODERATE | HIGH RISK | Measured | Rating |
|---|---|---|---|---|---|---|---|
| H1 | Untested Critical Logic | Critical modules with zero tests | 0 | 1–3 | >3 | 6+ (RealTimeTestService, MongoService, LegacyDataMapper, ConnectController, DiscoveryController, dev-api/realtime.js) | HIGH RISK |
| H2 | Low Test Coverage | Overall coverage % | >80% | 50–80% | <50% | ~2% (est., no coverage tool) | HIGH RISK |
| H3 | Missing Integration Tests | Boundaries covered % | >70% | 30–70% | <30% | 0% (no Feature/HTTP/DB/SSE tests) | HIGH RISK |
| H4 | Missing Contract Tests | APIs with contract tests % | >80% | 40–80% | <40% | 0% (~18 endpoints, 0 covered) | HIGH RISK |
| H5 | Flaky / Skipped Tests | Skipped/flaky test count | 0 | 1–5 | >5 | 1 (non-deterministic `random_int` test) | MODERATE |
| H6 | No CI Test Gate | Tests enforced in CI | Required gate | Runs, not required | No CI test run | Backend phpunit runs on PR (not proven required); frontend runs no tests | MODERATE |
| H7 | No End-to-End Tests *(additional)* | Critical user flows with E2E coverage % | >60% | 20–60% | <20% | 0% (no Cypress/Playwright) | HIGH RISK |
| H8 | Assertion-Free / Vacuous Tests *(additional)* | Tests with meaningful assertions % | >90% | 50–90% | <50% | 0% (0 of 2 assert real logic) | HIGH RISK |

## 5.4 Actions Required

| Hotspot | Action | Rating | Priority |
|---|---|---|---|

| | | | |
|---|---|---|---|
| H1 Untested Critical Logic | Add PHPUnit unit tests for `RealTimeTestService`, the reachability calc, `LegacyDataMapper`, `MongoService`; mirror in `dev-api` | HIGH RISK | CRITICAL |
| H2 Low Test Coverage | Enable coverage tooling per layer; drive critical modules to 75–80% before refactors | HIGH RISK | CRITICAL |
| H3 Missing Integration Tests | Add Laravel feature tests ( `RefreshDatabase` ) + Mongo round-trip ( `mongodb-memory-server` ) + SSE smoke tests | HIGH RISK | HIGH |
| H4 Missing Contract Tests | Add `assertJsonStructure` tests for all ~18 API routes; validate frontend client against `types/index.ts` | HIGH RISK | HIGH |
| H7 No End-to-End Tests | Add Playwright happy-path specs per module (discovery + connect flows), run against `dev-api` in CI | HIGH RISK | HIGH |
| H8 Vacuous Tests | Replace both tautological tests with real assertions; add mutation testing (Infection) | HIGH RISK | HIGH |
| H5 Flaky Test | Delete the `random_int` reachability test; ban randomness in `tests/`; replace with deterministic case | MODERATE | MEDIUM |
| H6 No CI Test Gate | Add frontend test step, enable coverage, make test jobs required branch-protection checks | MODERATE | MEDIUM |

## 5.5 Expected Outcomes

- **Critical telecom logic is protected before refactors** — IVR traversal, reachability scoring, and the unsafe `LegacyDataMapper` gain real unit tests, so extraction/refactor work has a behavioural safety net.
- **Measurable coverage replaces guesswork** — per-layer coverage instruments turn "~2% estimated" into an enforced, ratcheting number on the path to 75–80%.
- **Boundaries are verified, not assumed** — integration tests exercise the controller → MariaDB → Mongo → SSE wiring, catching failures that unit tests miss.
- **API contracts can't drift silently** — contract tests on all ~18 endpoints (and the frontend client) fail the build the moment a response shape changes, protecting the customer dashboard.
- **The frontend gains a safety net** — Vitest unit tests plus Playwright E2E cover the React flows and live SSE feed that today have zero automated verification.
- **CI becomes a real gate** — a required, coverage-aware check across backend, frontend, and E2E means regressions are blocked at merge instead of discovered in production.

The complete report (including §5.2 Hotspot-by-Hotspot Evidence with `affected-files` directives and §5.3 Mermaid diagrams) is saved at `docs/discovery/05-testing-and-quality-assurance.md`, ready for the UI to render to PDF.

---

## 2. Security Analysis

OVERALL CODEBASE RATING — SECURITY

**High Risk**

> Driven by a Critical missing-authentication gap across the entire API, wildcard CORS, PHP extract() variable injection, and committed default credentials with debug mode enabled.

**EXECUTIVE SUMMARY**

This review covered all three layers — the Laravel backend, the React/TypeScript frontend, and the Express `dev-api` reference server. The dominant, systemic finding is that **the entire API surface is unauthenticated and unauthorized**: every Discovery, Connect, dashboard, MongoDB, and legacy-report route in both `backend/routes/api.php` and `dev-api/src/server.js` is anonymously reachable, and reads/mutations use client-supplied IDs with no ownership checks. On top of that, CORS is wildcard-open (`allowed_origins => ['*']` and bare `cors()`), `LegacyReportController` runs PHP `extract()` over raw `$request->all()` (variable injection), the `dev-api` transcript query is exposed to MongoDB operator injection, default database credentials (`secret`/`root`) and `APP_DEBUG=true` are committed to `.env.example`, `docker-compose.yml`, and CI, and no rate limiting exists on expensive `start` / `run-check` / `bulk-import` endpoints. The **frontend is comparatively clean** — React auto-escaping is used throughout, there are no `dangerouslySetInnerHTML`/`innerHTML`/`eval` sinks, no client-side secrets, and no auth tokens in browser storage — its one real gap is the absence of a Content-Security-Policy. **Dependencies are current** (Laravel 12, React 19.2, Express 4.21, mongodb 6/2) with no known-EOL majors in the manifests. Because at least one unresolved Critical (no authentication) and multiple High findings exist, the overall security rating is **High Risk**.

## 6.1 Security Benchmark Ratings

| # | Security KPI | Target | GOOD | MODERATE | HIGH RISK | Measured | Rating |
|---|---|---|---|---|---|---|---|
| H1 | Critical Vulnerabilities | 0 | 0 | 1 | >1 | 1 (unauthenticated API surface) | MODERATE |
| H2 | High Vulnerabilities | 0 | <5 | 5–10 | >10 | 3 (CORS, extract() injection, misconfig) | GOOD |
| H3 | Medium Vulnerabilities | low | <20 | 20–50 | >50 | 5 | GOOD |
| H4 | Vulnerability Density | <0.5/KLOC | <0.5 | 0.5–1.0 | >1.0 | ≈3.1/KLOC (9 findings / ≈2.9 KLOC) | HIGH RISK |
| H5 | OWASP Top 10 Compliance | >95% | >95% | 80–95% | <80% | ≈22% categories clean | HIGH RISK |
| H6 | Critical/High Vulnerable Deps | 0 | 0 | 1 | >1 | 0 (manifest-based; scanner not run) | GOOD |
| H7 | Outdated Dependencies | <10% | <10% | 10–25% | >25% | <10% (current majors) | GOOD |
| H8 | End-of-Life Dependencies | 0 | 0 | 1–5 | >5 | 0 | GOOD |

## 6.5 Actions Required

| Finding | Action | Rating | Priority |
|---|---|---|---|
| Missing authentication & authorization (whole API) | Add Sanctum/JWT auth + ownership policies to all non-public routes in both APIs; gate/remove `bulk-import` | HIGH RISK | CRITICAL |
| Identification & Authentication Failures (6.7) | Introduce login, session/JWT lifecycle, MFA option, and brute-force lockout | HIGH RISK | CRITICAL |
| PHP `extract()` variable injection (6.3) | Replace `extract()` with validated explicit access in `LegacyReportController` & `LegacyDataMapper`; ban via PHPStan | MODERATE | HIGH |
| Wildcard CORS (6.5) | Replace `*` with explicit origin allow-list in `config/cors.php` and `dev-api` `cors()` | MODERATE | HIGH |
| Debug mode + committed default credentials (6.2/6.5) | Disable debug in prod, move secrets to a secret store, rotate exposed creds | MODERATE | HIGH |
| MongoDB operator injection (6.3) | Coerce/allow-list `module`, add `express-mongo-sanitize` in `dev-api` | MODERATE | MEDIUM |
| No rate limiting (6.4) | Add `throttle` (Laravel) / `express-rate-limit` + body-size caps | MODERATE | MEDIUM |
| Missing security headers & CSP (6.5 / FS5) | Add CSP/HSTS/X-Frame-Options at nginx edge and a CSP in `index.html` | MODERATE | MEDIUM |
| No security logging/monitoring (6.9) | Add structured audit logging for access/auth events + alerting | MODERATE | MEDIUM |
| No SAST/dependency scan in CI (6.12) | Add `composer audit` + `npm audit` + a SAST step and secret scanning to `ci.yml` | MODERATE | MEDIUM |

The full report — including §6.2 hotspot evidence with `affected-files` directives and the §6.4 Mermaid diagrams — is written to `docs/discovery/06-security.md` and will be rendered to `docs/discovery/06-security.pdf` by the orchestration UI.

---

## 3. Performance & Sustainability Analysis

OVERALL CODEBASE RATING — PERFORMANCE & SUSTAINABILITY

# High Risk

Driven by P4 Memory Efficiency (uncleaned interval leak + unbounded in-memory store + full-collection loads) and P10 Build Efficiency (no dependency/layer caching, pecl recompiled every CI run); multiple Moderate hotspots across algorithms, API, network, and concurrency compound the verdict.

EXECUTIVE SUMMARY

The Klearcom platform is a small full-stack voice-QA monolith whose runtime-performance health is **moderate overall but pulled to High Risk by two clear drivers**: an unbounded-memory lane (a deliberately un-cleaned polling interval in `LegacyMonitorPoller`, ever-growing in-memory arrays in the dev-API store, and full-collection `iterator_to_array` loads on every Mongo read) and a build pipeline with **zero dependency/layer caching** that recompiles the PHP `mongodb` extension via `pecl` on every CI run. On the algorithm layer, three copies of a recursive `buildTree` re-filter the entire node collection at every level (O(n²)), and reachability math is duplicated across four call sites. The API layer streams Server-Sent Events by re-reading and re-serializing the *entire* event collection every 500 ms, while the frontend simultaneously runs three React-Query polling loops per page — chatty, duplicative traffic that nginx serves without gzip/brotli compression. Synthetic `usleep()` delays hold a PHP-FPM worker for 2.5–3.6 s per test, and always-on containers carry no resource limits or right-sizing. Database N+1 and cache-layer findings are deferred to Backend Modernization (H14/H10) to avoid conflicting counts. The dominant risk sits in the **memory + build + network layers**, not raw CPU.

## 7.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | GOOD | MODERATE | HIGH RISK | Measured |
|---|---|---|---|---|---|---|
| P1 | Algorithm Efficiency | High-complexity algorithm sites | 0 | 1–5 | >5 | 5 (3× recursive `buildTree` O(n²) - array filter + per-row collection scan) |
| P2 | Database Performance | Deferred → Backend Modernization (H14/H10) | — | — | — | See Backend Modernization |
| P3 | API Performance | Response-latency hotspots | 0 | 1–5 | >5 | 5 (2× unbounded list `->get()`, SS loop, sequential Mongo+SQL fan-ou monitor report) |
| P4 | Memory Efficiency | High-memory sites | 0 | 1–3 | >3 | 4 (interval leak, unbounded dev store `iterator_to_array` full load, un `>get()`) |
| P5 | CPU Efficiency | CPU-intensive operations | 0 | 1–5 | >5 | 2 (repeated full-list `json_encode` / `JSON.stringify` loop) |
| P6 | Concurrency | Parallelizable work + pool sizing (blocking-I/O → Backend Modernization H14) | 0 | 1–5 | >5 | 2 ( `new Client` per request — no pooling/singleton; `afterResponse` work holds one FPM worker) |
| P7 | Caching | Deferred → Backend Modernization H14 / Frontend Modernization H11 | — | — | — | See those reports |
| P8 | Resource Utilization | Over-provisioned / idle resources | 0 | 1–3 | >3 | 3 (no CPU/memory limits on any cor always-on services, Vite dev-server frontend image) |
| P9 | Network Efficiency | Excessive-traffic sites | 0 | 1–5 | >5 | 5 (2 pages × 3 concurrent poll loops, poller, SSE+polling duplication, no g |

| | | | | | | |
|---|---|---|---|---|---|---|
| P10 | Build Efficiency | Build/test pipeline efficiency | efficient | partial | slow / no caching | No composer/npm/layer caching; `p` `mongodb` recompiled every run |
| P11 | Logging Efficiency | Excessive-logging sites | 0 | 1–10 | >10 | 0 (only boot-time `console.log` ar `.catch(console.error)` ; none |
| P12 | Sustainability | Resource-optimization posture | optimized | partial | wasteful | partial (always-on + no right-sizing + synthetic delays + chatty polling; foo no carbon awareness) |
| P13 | Blocking Synthetic Delays *(additional)* | Blocking `sleep()` / `usleep()` on a request/worker path (0 · 1–2 · >2) | 0 | 1–2 | >2 | 2 ( `RealTimeTestService::runDis` ≈3.6 s, `runConnectTest` ≈2.5 s) |

## 7.5 Actions Required

| Hotspot | Action | Rating | Priority |
|---|---|---|---|
| P4 Memory Efficiency | Add `componentWillUnmount` cleanup to `LegacyMonitorPoller` , cap/stream Mongo reads ( `getTestEvents` limit + lazy cursor), and bound the dev-API in-memory store | HIGH RISK | CRITICAL |
| P10 Build Efficiency | Add Composer/npm `actions/cache` and cache/prebuild the `mongodb` PECL extension; bake `vendor/` into the PHP image | HIGH RISK | HIGH |
| P1 Algorithm Efficiency | Replace recursive O(n²) `buildTree` with single-pass group-by and consolidate the three duplicates into one shared builder | MODERATE | HIGH |
| P3 API Performance | Make SSE reads incremental (since-cursor) and add pagination to `index` endpoints | MODERATE | HIGH |
| P9 Network Efficiency | Drive live UI from SSE and drop redundant poll loops; enable gzip/brotli in nginx | MODERATE | HIGH |
| P6 Concurrency | Register `MongoService` as a pooled singleton; move background tests to a sized queue worker | MODERATE | MEDIUM |
| P13 Blocking Synthetic Delays | Move `usleep` -driven simulation off the FPM worker onto an async queue | MODERATE | MEDIUM |
| P8 Resource Utilization | Add container resource limits and convert the frontend image to a multi-stage production build | MODERATE | MEDIUM |
| P5 CPU Efficiency | Serialize each SSE event once and reuse the cached string | MODERATE | LOW |
| P12 Sustainability | Adopt the P4/P9/P10/P13 fixes and add right-sizing + an efficiency KPI to cut compute/carbon | MODERATE | LOW |

## 7.6 Expected Outcomes

- **Lower-complexity algorithms:** a single-pass O(n) tree builder keeps `/tree` and IVR-depth reports flat-latency as node counts grow into the hundreds, and one shared implementation eliminates divergent copy-paste fixes.

- **Leaner memory footprint:** clearing the polling interval, capping/streaming Mongo cursor reads, and bounding the dev store remove the leak, GC pressure, and unbounded-growth risks that currently drive the High-Risk verdict.
- **Higher throughput, fewer round-trips:** incremental SSE reads plus dropping the triple polling loops cut per-test network/CPU work by 3–4× and free the browser from duplicating streamed data; nginx compression trims JSON/asset bytes on the wire.
- **Better capacity under load:** a pooled Mongo client and a real queue worker (instead of `afterResponse` + `usleep`) stop FPM workers from being parked asleep, restoring request capacity during concurrent tests.
- **Lower cost & carbon:** CI dependency/layer caching (no more per-run PECL compile), right-sized always-on containers, and a built-not-dev frontend image cut both cloud spend and energy per build/deploy.

---

## 4. Technical Debt

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

# Moderate

Driven by a missing composer.lock (non-reproducible backend builds), no enforced lint/style gate, no DB migration framework, and shallow/non-deterministic test coverage — no single dimension is High Risk, but every dimension carries real gaps.

EXECUTIVE SUMMARY

Klearcom is a well-structured polyglot monolith with genuinely strong agentic scaffolding: five `AGENTS.md` guides, a symmetric Discovery/Connect module layout, a working GitHub Actions CI pipeline, and Docker Compose for the full stack. That foundation makes it more agent-ready than most codebases of its age. The debt is concentrated in reproducibility and quality-gate depth rather than architecture. The three most severe gaps are: (1) **`backend/composer.lock` is not committed**, so every CI run and every developer install resolves PHP dependencies afresh — backend builds are not reproducible; (2) **no code-style enforcement exists** — no ESLint, Prettier, Laravel Pint, and although `phpstan/phpstan` is declared it has no config file and is never invoked in CI, so agent-authored code has no automated style/lint gate; (3) **the database has no migration framework** — the entire schema lives in a single `docker/mariadb/init.sql` that only executes on a fresh MariaDB volume, leaving no versioned, reversible path for schema evolution. There are also self-documented debts (a deliberately reintroduced `extract()` pattern, a non-deterministic test using `random_int()`, missing dev-api input validation and rate limiting) captured in `docs/CODEBASE_AUDIT_ISSUES.md`. Overall the repo is **Moderate** readiness: a real CI gate exists and can be trusted for the frontend, but the missing lock file, absent lint layer, and flaky/shallow tests mean agent-authored output cannot yet be verified with full confidence.

## Readiness Benchmark Ratings

| # | Dimension | GOOD | MODERATE | HIGH RISK | Measured | R |
|---|---|---|---|---|---|---|
| D1 | Code Repository | all checks pass | 1–2 gaps | 3+ gaps / no CI | `.gitignore` + CI + JS lock files present; `composer.lock` | |

| | | | | | | |
|---|---|---|---|---|---|---|
| | Health | | | | **missing**, no CODEOWNERS/PR template, CI runs no lint step | |
| D2 | Third-Party Tool Usage | mostly wired & current | some unused/unwired | many unused/unmaintained | 18/19 packages wired; `phpstan` declared but no config and never run | |
| D3 | AI Tool / Agentic Readiness | ready | partial | not ready | 5 `AGENTS.md` + `.kiro/` config + symmetric enumerable modules; but CI verification gate is shallow (no lint, one flaky test, dev-api untested) | |
| D4 | Database Usage | sound | some gaps | no constraints / shared flat schema | FKs with `ON DELETE CASCADE` + Mongo indexes present; **no migration framework**, few secondary indexes, schema+seed combined & non-idempotent | |
| D5 | Development Environment | reproducible | partial | manual / fragile | Docker Compose + Dockerfiles + backend/dev-api `.env.example`; **no lint/formatter enforced**, no `frontend/.env.example` | |
| D6 | Automated Quality Gates & Test Determinism *(additional)* | lint+type+deterministic tests gate every PR | partial gate | no gate / flaky | CI runs `phpunit` + `tsc --noEmit` + `vite build`; but no lint, no coverage, `ReachabilityCalculationTest` uses `random_int()` (flaky), dev-api has zero tests | |

**Additional readiness gaps:** one additional dimension (**D6 — Automated Quality Gates & Test Determinism**) was observed beyond the five standard dimensions, driven by the flaky `random_int()` test and the absence of any lint step in CI. No further readiness gaps beyond these were observed.

## 8.8 Actions Required

| Gap | Action | Rating | Priority |
|---|---|---|---|
| `backend/composer.lock` not committed — non-reproducible backend builds | Run `composer update` and commit `composer.lock`; keep it in sync going forward | MODERATE | CRITICAL |
| No code-style / static-analysis enforcement | Add ESLint + Prettier (frontend), Laravel Pint + a `phpstan.neon` (backend); wire all into CI and a pre-commit hook | MODERATE | HIGH |
| CI gate too shallow to accept agent output | Add a lint/static-analysis job, a `dev-api` test job, and make `ReachabilityCalculationTest` deterministic (remove `random_int()`) | MODERATE | HIGH |
| No database migration framework — schema only applies on fresh volume | Adopt Laravel migrations (or a versioned SQL tool); back-fill current schema as migration 0001; add rollback path | MODERATE | HIGH |
| `phpstan` declared but never configured or run | Add `phpstan.neon` and a `phpstan analyse` CI step, or remove the dependency | MODERATE | MEDIUM |

| | | | |
|---|---|---|---|
| No branch-protection signals (CODEOWNERS, PR template) | Add `CODEOWNERS` per top-level module and `.github/PULL_REQUEST_TEMPLATE.md`; require CI checks on `main` | MODERATE | MEDIUM |
| Missing secondary DB indexes on FK / status / lookup columns | Add indexes on `discovery_nodes(discovery_job_id, parent_id)`, `connect_check_results(connect_monitor_id)`, and `*.status` | MODERATE | MEDIUM |
| Schema DDL and seed data combined; container seeds non-idempotent | Split DDL from seed data; guard seed inserts behind existence checks | MODERATE | LOW |
| No `frontend/.env.example`; `frontend/Dockerfile` uses `npm install` not `npm ci` | Add `frontend/.env.example` with `VITE_API_URL`; switch Dockerfile to `npm ci` | MODERATE | LOW |

The Markdown deliverable is written; the orchestration UI will render `docs/discovery/08-technical-debt.pdf` from it.